

Lecture 7

- local linear perspective
 - Sign SGD
 - Gradient Descent
 - Shampoo
 - Muon
-

Why do we care about optimizers?

• Want to train fast. Training time = \$\$\$.

• AdamW + tuned hyperparameters are default.

• Hyperparameter search is difficult for new optimizers.

→ But under Adam we see lazy training → i.e. models barely change from their random initialization

→ Initialization rules are nice.
→ Before this everything was $N(0,1)$

$\vec{\theta}$: parameters.

$l(\vec{x}, \vec{\theta})$ is the loss.

$\vec{x}$: data.

Linearized perspective:

$$l(\vec{x}, \vec{\theta}_i + \Delta \vec{\theta}) \approx l(\vec{x}, \vec{\theta}_i) + \left. \frac{\partial l}{\partial \theta} \right|_{\vec{\theta} = \vec{\theta}_i} \Delta \vec{\theta} + \dots$$

Gradient Descent takes a step.

→ Want this to be small enough that the approximation still holds

→ Want to converge fast

~~$$\frac{1}{2} (\vec{x}_i^T \vec{\theta} - y)^2$$~~

$$\nabla_{\theta} = (\vec{x}_i^T \theta - y) \cdot \vec{x}_i$$

SGD-LS: $\vec{\theta}_{t+1} = \vec{\theta}_t - \eta (\vec{x}_i^T \vec{\theta}_t - y) \cdot \vec{x}_i$

SGD: $\theta_{t+1} = \theta_t - \eta \cdot \left. \nabla_{\theta} l(\vec{x}, \vec{\theta}) \right|_{\vec{\theta} = \theta_t, \vec{x} = \vec{x}_i}$

Follow perspective from Bernstein + Newhouse 2024.

↳ Build on many other works

$$\mathcal{L}(\vec{\theta}) = \frac{1}{n} \sum_{j=1}^n \ell(\vec{x}_j, \vec{\theta})$$

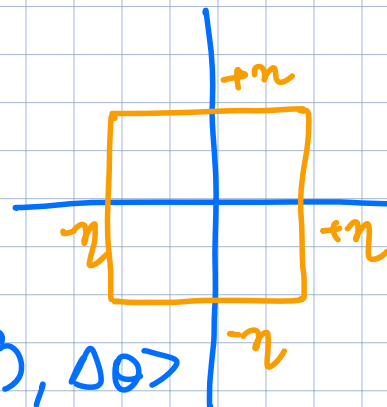
Avg. loss over data points.

Want:

$$\underset{\|\vec{\Delta\theta}\| \leq \eta}{\operatorname{argmin}} \left(\mathcal{L}(\vec{\theta}) + \langle \nabla_{\vec{\theta}} \mathcal{L}(\vec{\theta}), \vec{\Delta\theta} \rangle \right)$$

$$= \underset{\|\vec{\Delta\theta}\| \leq \eta}{\operatorname{argmin}} \langle \nabla_{\vec{\theta}} \mathcal{L}(\vec{\theta}), \vec{\Delta\theta} \rangle.$$

Choice 1: $\|\vec{\Delta\theta}\|_{\infty} \leq \eta \rightarrow$ Linear program



$$\rightarrow \underset{\|\vec{y}\|_{\infty} \leq \eta}{\operatorname{argmin}} \vec{x}^T \vec{y}$$



$$\underset{\|\vec{\Delta\theta}\|_{\infty} \leq \eta}{\operatorname{argmin}} \langle \nabla_{\vec{\theta}} \mathcal{L}(\vec{\theta}), \vec{\Delta\theta} \rangle$$

Solution:

$$\Delta\theta[j] = \begin{cases} +\eta & \text{if } \nabla_{\theta} \mathcal{L}(\vec{\theta})[j] < 0 \\ -\eta & \text{if } \nabla_{\theta} \mathcal{L}(\vec{\theta})[j] > 0 \end{cases}$$

$$\Rightarrow \vec{\Delta\theta} = \underbrace{-\eta \cdot \text{sgn}(\nabla_{\theta} \mathcal{L}(\vec{\theta}))}_{\text{sign SGD}}.$$

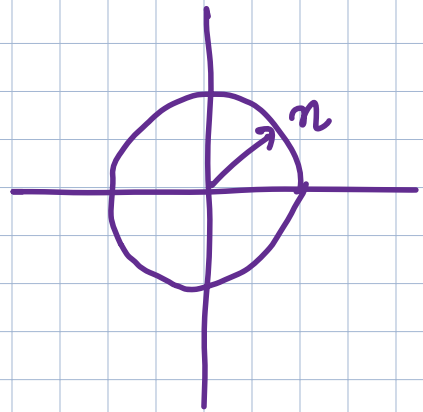
sign SGD.

Choice 2:

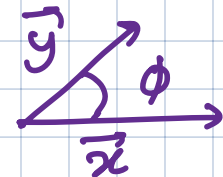
$$\|\vec{\Delta\theta}\|_2 \leq \eta.$$

$$\langle \nabla_{\theta} \mathcal{L}(\vec{\theta}), \vec{\Delta\theta} \rangle.$$

$$\underset{\|\Delta\theta\|_2 \leq \eta}{\text{argmin}}$$



$$\vec{x}^T \vec{y} = \|\vec{x}\|_2 \|\vec{y}\|_2 \cos \phi$$



Cauchy-Schwartz: Inner product absolute value maximized when vectors are aligned.

$$\Rightarrow \Delta \vec{\theta} = -\eta \underbrace{\frac{\nabla_{\theta} \mathcal{L}(\vec{\theta})}{\|\nabla_{\theta} \mathcal{L}(\vec{\theta})\|_2}}_{\text{Unit norm.}}$$

scale by η .

Alternatively, take the regularization / Lagrange multiplier perspective.

Optimize $\underbrace{\vec{\theta}}_{\text{unconstrained.}}$

$$\underbrace{\langle \nabla_{\theta} \mathcal{L}(\vec{\theta}), \Delta \theta \rangle + \lambda \|\Delta \theta\|_2^2}_{g(\Delta \theta). \text{ convex.}}$$

→ Sweeps out the same family of solutions as constrained version.

$$\nabla g(\Delta \theta) = \nabla_{\theta} \mathcal{L}(\vec{\theta}) + 2\lambda \Delta \vec{\theta} \rightarrow \text{Set to zero.}$$

$$\text{Minimize @ } \Delta \vec{\theta}^* = -\underbrace{\frac{1}{2\lambda}}_{\text{stepsize}} \underbrace{\nabla_{\theta} \mathcal{L}(\vec{\theta})}_{\text{gradient descent}}$$

Linearization around matrices?

$\vec{\theta}$: we've been thinking of this as a vector.

But neural network parameters are naturally organized in matrices.

$$\vec{h}_1 = \text{ReLU}[\vec{b}_1 + \boxed{W_0 \vec{x}}]$$

$$\vec{h}_j = \text{ReLU}[\vec{b}_j + W_{j-1} \cdot \vec{h}_{j-1}]$$

$$y = \text{ReLU}[\vec{b}_k + W_k \vec{h}_k]$$

Recall from Xavier initialization ... scaling problems come from the weight matrices.

So what if we think about change ΔW in "weight matrix space" instead.

$$\begin{array}{l} \text{argmin.} \\ \|\Delta W\| \leq \eta \end{array} \quad \underbrace{\langle \nabla_w \mathcal{L}(w), \Delta W \rangle}_{\text{matrix} \quad \text{matrix.}}$$

Recall: $\langle A, B \rangle_F = \text{trace}(A^T B)$

Consider: $\|\Delta W\|_2 \leq \eta$ ie. spectral norm.

Recall: $\|A\|_2 = \sigma_{\max}$ largest singular value

$$\max_{\|\vec{x}\|_2=1} \|A\vec{x}\|_2 = \|A\|_2 = \sigma_{\max}$$

$$\underset{\|\Delta W\|_2 \leq \eta}{\text{argmin}} \quad \langle \nabla_w \mathcal{L}(w), \Delta W \rangle.$$

$$\text{let } \nabla_w \mathcal{L}(w) = U \Sigma V^T.$$

$$\text{Then } \Delta W^* = -\eta U V^T.$$

This gives us the Shampoo Optimizer (Gupta 2017/2018)

→ A variant of Shampoo (Dahl 2023) won AlgoPerf
→ Training alg. competition.

$$\max_{\|B\|_2 \leq 1} \text{trace}(A^T B)$$

$$\text{trace}(ABC) \\ = \text{trace}(BCA)$$

$$\text{Let } A = U \Sigma V^T \\ = \sum_i \sigma_i \vec{u}_i \vec{v}_i^T$$

$$A = U \Sigma V^T$$

$$\max \text{trace}(A^T B)$$

$$\|B\|_2 \leq 1$$

$$= \max_{\|B\|_2 \leq 1} \text{trace}(V \Sigma^T U^T B)$$

$$\|B\|_2 \leq 1$$

$$= \max_{\|B\|_2 \leq 1} \text{trace} \left(\sum_i \sigma_i \vec{u}_i \vec{u}_i^T B \right) \quad \text{Trace is linear.}$$

$$= \max_{\|B\|_2 \leq 1} \sum_i \text{trace}(\sigma_i \vec{u}_i \vec{u}_i^T B)$$

$$= \max_{\|B\|_2 \leq 1} \sum_i \text{trace} \left(\sigma_i \underbrace{\vec{u}_i^T B \vec{u}_i}_{\text{Scalar}} \right)$$

$$= \max_{\|B\|_2 \leq 1} \sum_i \sigma_i \underbrace{\vec{u}_i^T B \vec{u}_i}_{\text{Scalar}}$$

$$\|B \vec{u}_i\|_2 \leq \|\vec{u}_i\|_2 \cdot \|B\|_2 \leq 1.$$

$$\|\vec{u}_i^T B \vec{u}_i\|_2 \leq 1. \quad \text{Cauchy-Sch.}$$

$$\leq \sum_i \sigma_i$$

→ To attain bound $B = U V^T$

$$\begin{aligned} & \text{trace}(V \Sigma^T U^T U V^T) \\ &= \text{trace}(\Sigma^T) \end{aligned}$$

Beinstein + Newhouse 2024

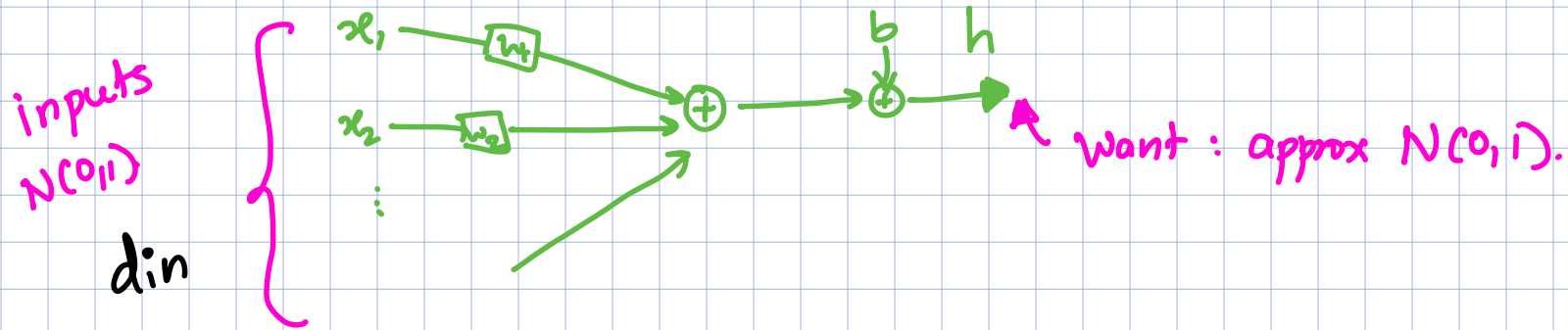
- Choose a norm
- Choose a stepsize.

→ The right norm may depend on geometry / architecture of network.

- But we have a recipe
 - Given a norm, make an optimizer.
 - examples for $\|\cdot\|_2$, $\|\cdot\|_\infty$, spectral norm.

So what might be a good norm for deep learning?

Recap Xavier initialization.



Can we rewrite this in terms of norms?

What norm are we trying to preserve?

$$h = \vec{w}^T \vec{x} = \sum w_i x_i$$

$$E[w_i^2] = \sigma_w^2$$

$$w_i \text{ i.i.d. } E[w_i] = 0$$

$$E[h^2] = \sigma_w^2 [\sum x_i^2]$$

$$\text{if } x_i^2 \approx 1 \text{ we want } \sigma_w^2 \approx \frac{1}{d_{in}}$$

Consider RMS-norm:

$$\|\vec{x}\|_{RMS} = \frac{1}{\sqrt{d_{in}}} \|\vec{x}\|_2 = \frac{1}{\sqrt{d_{in}}} \sqrt{\sum_i x_i^2}$$

If $\|\vec{x}\|_{RMS} = 1$
then every entry of
 $\vec{x}, x_i] \approx 1 \forall i$

RMS-norm $\|\vec{x}\|_{\text{rms}} = 1 \rightarrow$ every entry is approximately 1!

This norm is naturally presented in Xavier initialization

Hmm... So maybe RMS norm is interesting for DNNs.
Is there a matrix version? (Recall Shampoo...)

Recall: induced Matrix-norm

$$\|A\|_{\alpha \rightarrow \beta} = \max_{\|\vec{x}\|_{\alpha}=1} \|A\vec{x}\|_{\beta}$$

RMS $\rightarrow$ RMS norm?

$$\begin{aligned} \|A\|_{\text{RMS} \rightarrow \text{RMS}} &= \max_{\|\vec{x}\|_{\text{RMS}}=1} \|A\vec{x}\|_{\text{RMS}} \\ A \quad \begin{array}{|c|} \hline d_1 \\ \hline \end{array} \quad \begin{array}{|c|} \hline d_2 \\ \hline \end{array} &= \max_{\|\vec{x}\|_2 = \sqrt{d_1}} \frac{1}{\sqrt{d_2}} \cdot \|A\vec{x}\|_2 \end{aligned}$$

$$= \frac{\sqrt{d_1}}{\sqrt{d_2}} \cdot \max_{\|\vec{x}\|_2=1} \|A\vec{x}\|_2$$

Spectral norm!

So can we use this induced RMS $\rightarrow$ RMS norm in our optimizer recipe?

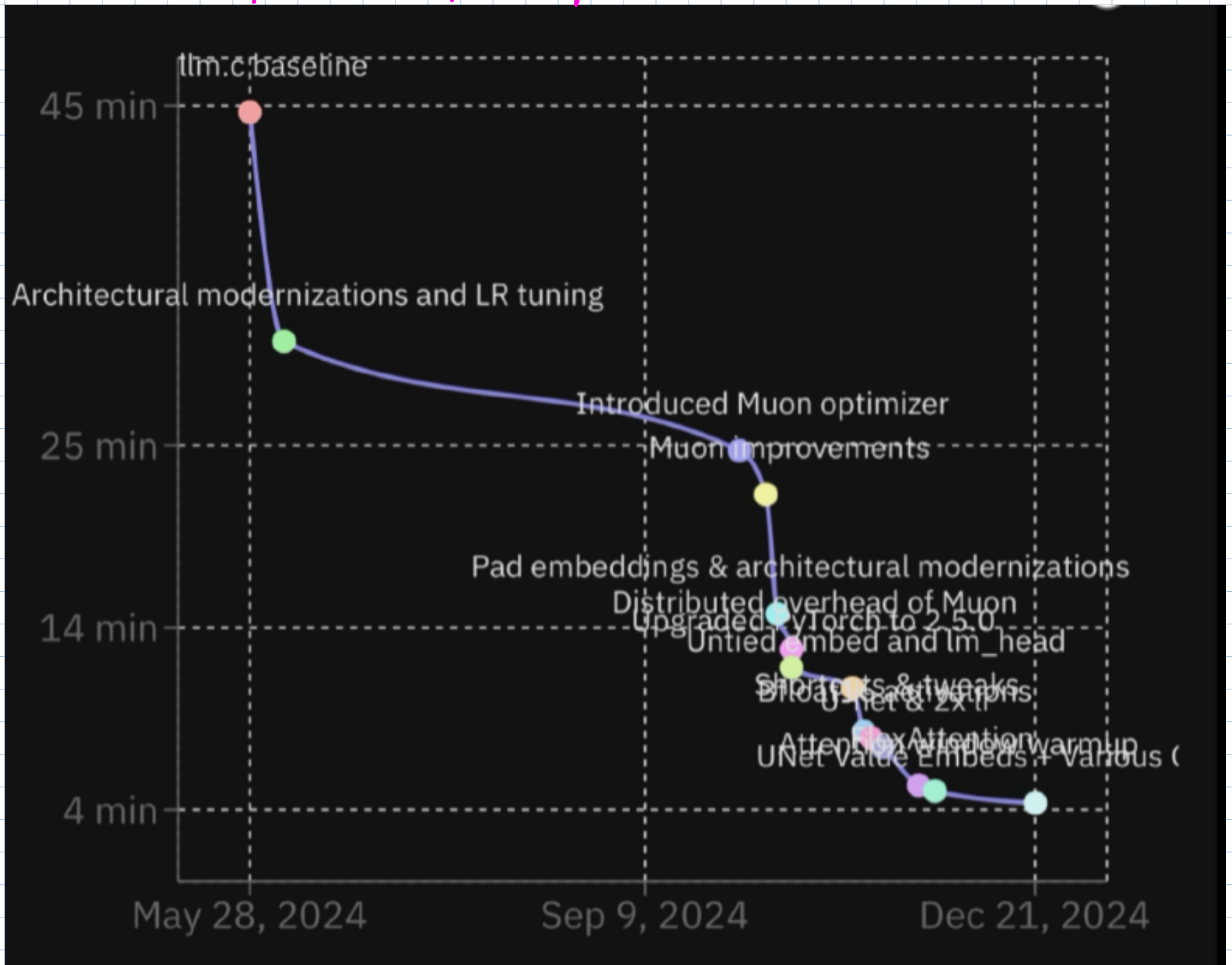
$$\|\Delta W\|_{\text{RMS}^*} \leq \eta?$$

Key observation: If we choose this norm, we can choose only one hyperparameter η across all layers, but the effect will be layer specific learning rates, due to fan-in and fan-out.

$$\begin{aligned}\|\Delta W\|_{\text{RMS}^*} \leq \eta &\Rightarrow \sqrt{\frac{d_1}{d_2}} \|\Delta W\|_2 \leq \eta. \\ &\Rightarrow \|\Delta W\|_2 \leq \eta \sqrt{\frac{d_1}{d_2}}\end{aligned}$$

$\rightarrow$ Essential idea behind mup $\rightarrow$ maximal update parameterization.

Nano GPT speedrun.



Optimizer Recipe w/ RMS norm

What if we had the RMS-RMS norm instead? $W \in \mathbb{R}^{d_{out} \times d_{in}}$

$$\underset{\|\Delta W\|_{\text{RMS} \rightarrow \text{RMS}} \leq \eta}{\text{argmin}} \quad \langle \nabla_W \mathcal{L}(W), \Delta W \rangle$$

$$= \underset{\|\Delta W\|_2 \leq \sqrt{\frac{d_{out}}{d_{in}}} \cdot \eta}{\text{argmin}} \quad \langle \nabla_W \mathcal{L}(W), \Delta W \rangle$$

$$\Rightarrow \Delta W^* = -\eta \cdot \sqrt{\frac{d_{out}}{d_{in}}} U_r V_r^T$$

Muon key idea 1.

Why might semi-orthogonal matrices be a good idea?

→ Condition number $U_r V_r^T = 1$. Uniform step in all directions.

→ Not dominated by largest singular values.

But computing UV^T is expensive.

→ Needs SVD.

Muon solves this issue through two observations

(1) Getting the direction approximately correct is good enough.

(2) Newton-Schulz iterations. ← Muon key idea 2.

$$\underline{U\Sigma V^T} \rightarrow \underline{UV^T}.$$

i.e. replace all the singular values by 1.

How?

$$f(U\Sigma V^T) \overset{\sim}{=} UV^T$$

Newton-Schulz.

① Odd polynomials commute with the SVD.

$$p(x) = a_0 x + a_1 x x^T x + a_2 (x x^T)^2 x + \dots + a_n (x x^T)^n x.$$

e.g. $p(x) = \frac{3}{2} \cdot x - \frac{1}{2} x x^T x$.

$$\begin{aligned} p(U \Sigma V^T) &= \frac{3}{2} U \Sigma V^T - \frac{1}{2} (U \Sigma V^T)(V \Sigma^T U^T) \cdot U \Sigma V^T \\ &= \frac{3}{2} U \Sigma V^T - \frac{1}{2} U \Sigma \Sigma^T \Sigma V^T \\ &= U \left[\frac{3}{2} \Sigma - \frac{1}{2} \Sigma \Sigma^T \Sigma \right] V^T \\ &= U p(\Sigma) \cdot V^T \end{aligned}$$

So $p(x) = U \underline{p(\Sigma)} V^T$.

So can apply to a matrix without changing the singular vectors,

② Can we find $p(x)$ such that
 $p(x) \rightarrow 1$ for $x > 0$

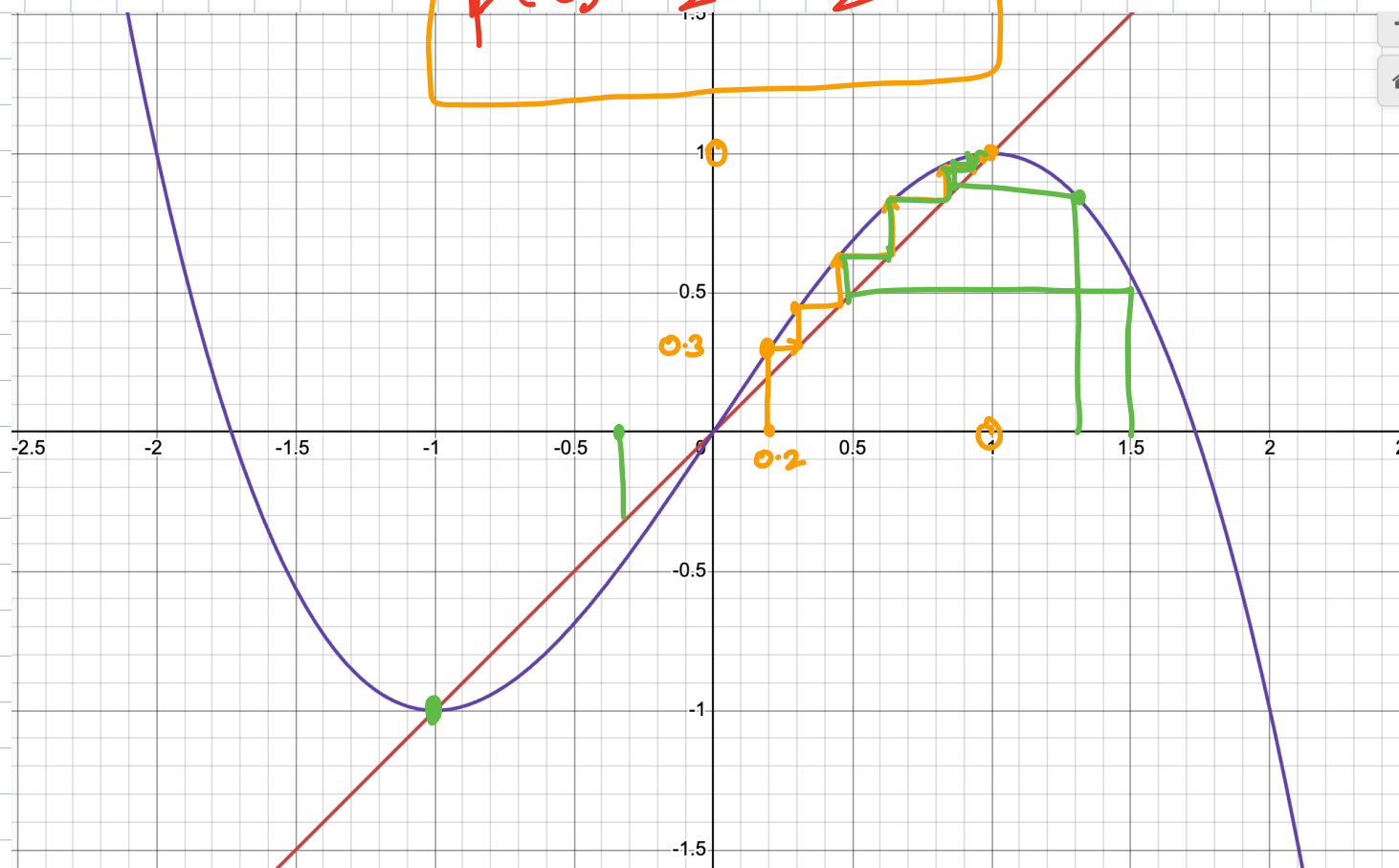
$$p(x) \rightarrow U p(\epsilon) V^T$$

$$p(p(x)) \rightarrow U p(p(\epsilon)) V^T$$

$$\vdots$$

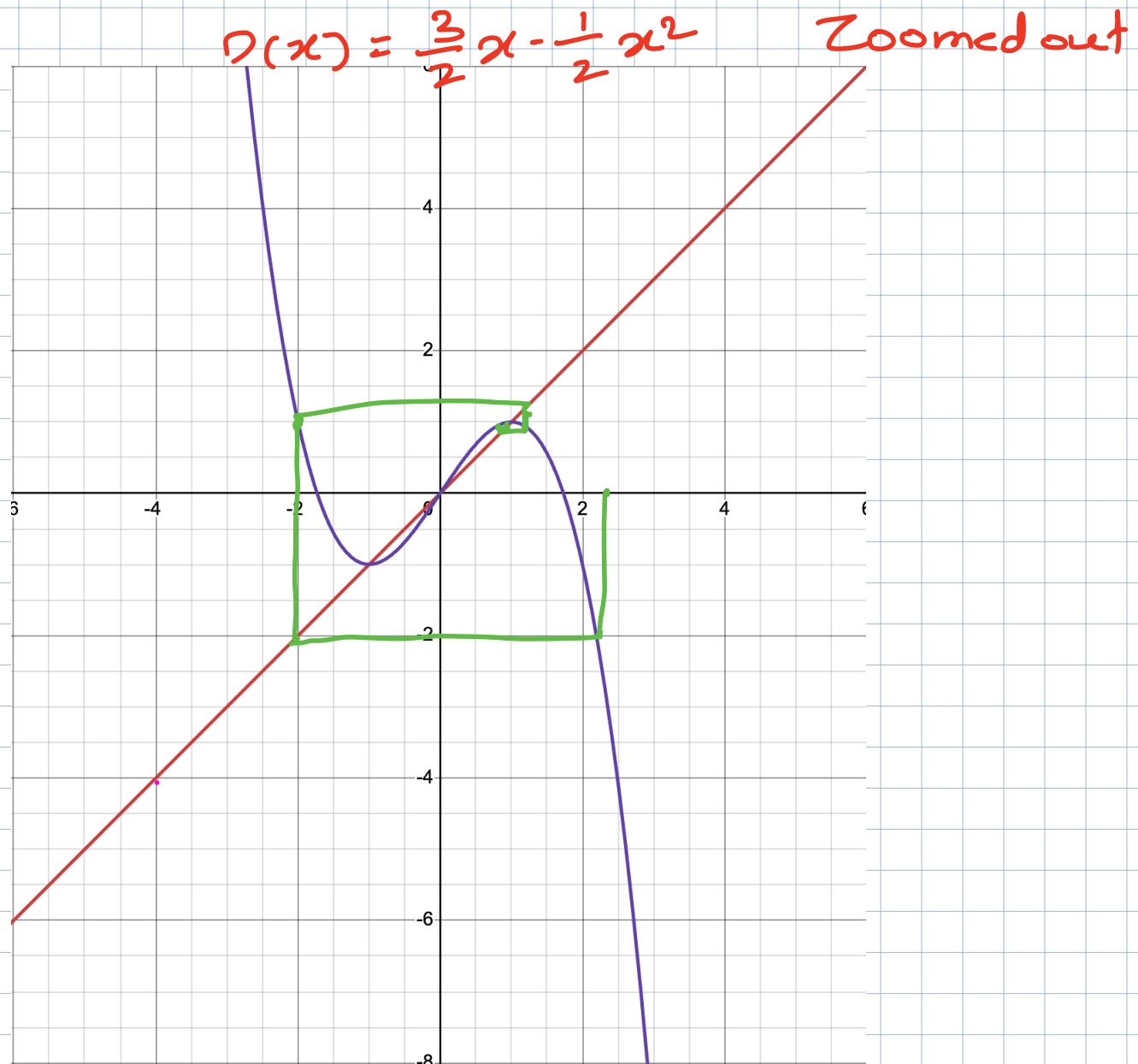
Then, iteratively apply p .

$$p(x) = \frac{3}{2}x - \frac{1}{2}x^3$$



$$p(p(p(p(\sigma)))) \rightarrow p^n(\sigma) \rightarrow 1$$

$$n \rightarrow \infty$$



→ Need to have singular values $\in (0, 1]$ before applying!

→ How? Normalize by the Frobenius norm : $\frac{G}{\|G\|_F}$.